Stack and Queue

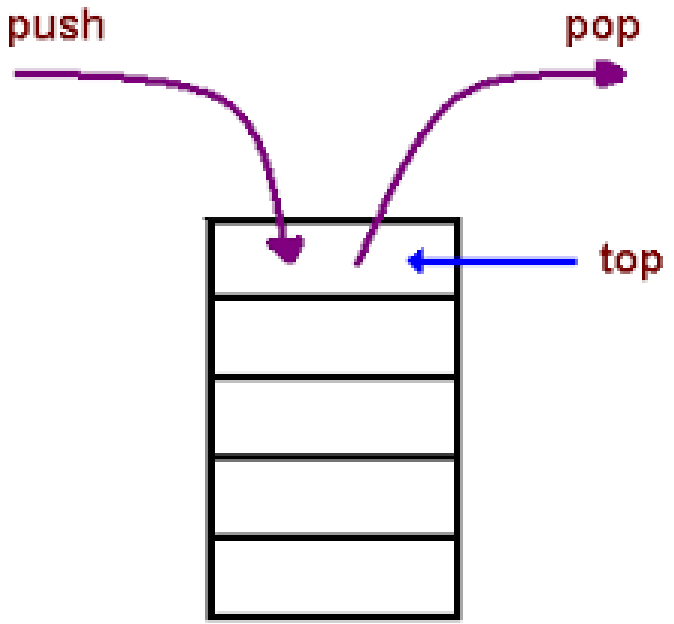
- Definition of a Stack
- Operations with a Stack
- Array and Stack
- Implementation of a Stack
- Application of Stack (Arithmetic Expression Conversion and Evaluation)
- Definition of a Queue
- Operations with a Queue
- Array, Stack and Queue
- Implementation of a Queue

STACK

- A stack or LIFO (last in, first out) is an abstract data type that serves as a collection of elements, with two principal operations:
  - **push** adds an element to the collection on top;
  - **pop** removes the last (top of the stack) element that was added.
- Bounded capacity
  - If the stack is full and does not contain enough space to accept an entity to be pushed, the stack is then considered to be in an overflow state.
  - A pop either reveals previously concealed items or results in an empty stack – which means no items are present in stack to be removed.
- Non-Bounded capacity
  - Dynamically allocate memory for stack. No overflow.

STACK

Stack can be represented by an *array (stack)* with an index *Top*



Top usually starts from 0 when no element is in stack [empty condition]

Top is increased by 1 if an element is pushed

Top is decreased by 1 if an element is popped

Don't be confused between ***Top*** and ***value at top of the stack***

OPERATION ON STACK - PUSH

Algorithm:

Input: Array (stack), MaxSize, Top, item (to push)

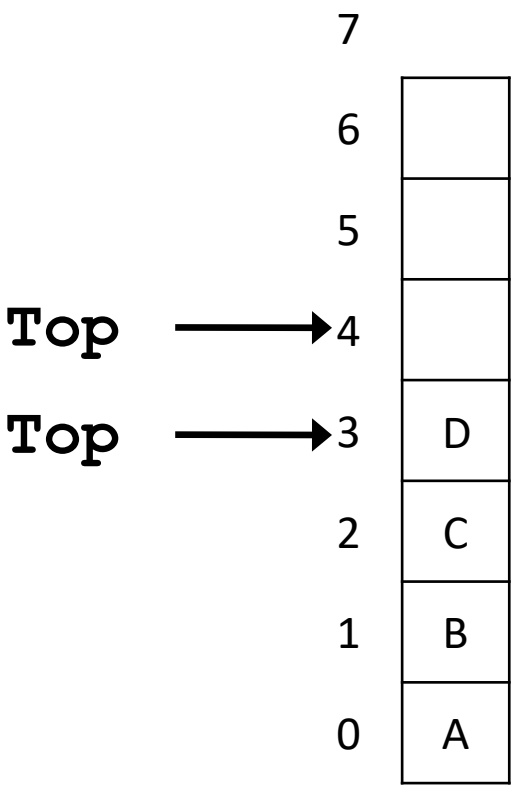
Step 1: if stack is full exit with status ‘unsuccessful push’

else

store *item* at position *Top*

Step 2: increase *Top* by 1

Push Operation



Considering **`MaxSize`** = 7
There are 3 elements inside Stack
So next element will be pushed at index 3

OPERATION ON STACK - POP

Algorithm:

Input: Array (stack), MaxSize, Top

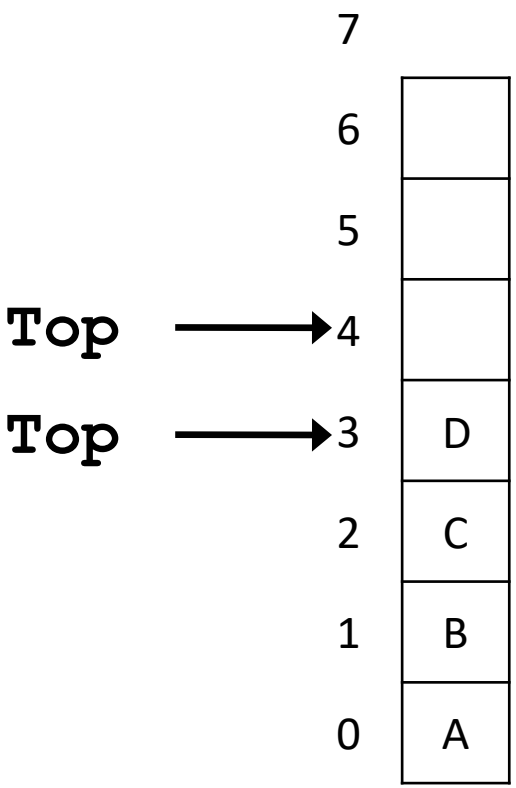
Step 1: if stack is empty exit with status ‘unsuccessful pop’

else

decrease *Top* by 1

Step 2: store value at T*op* to a variable *item*

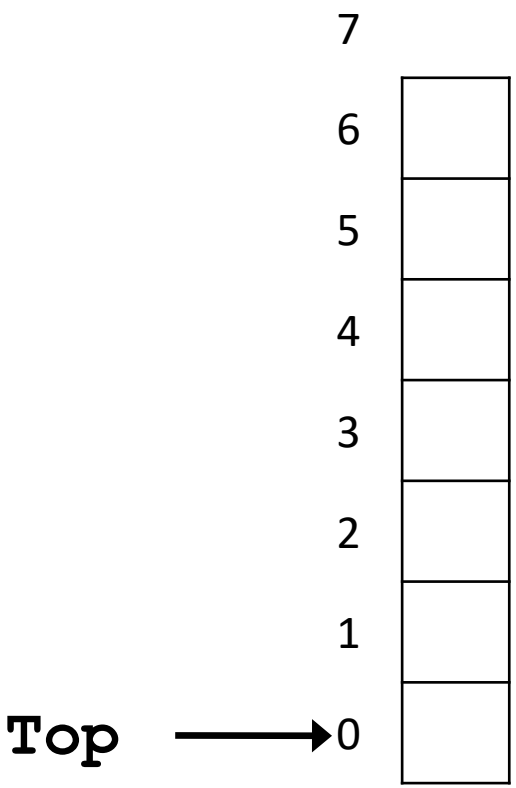
Pop Operation



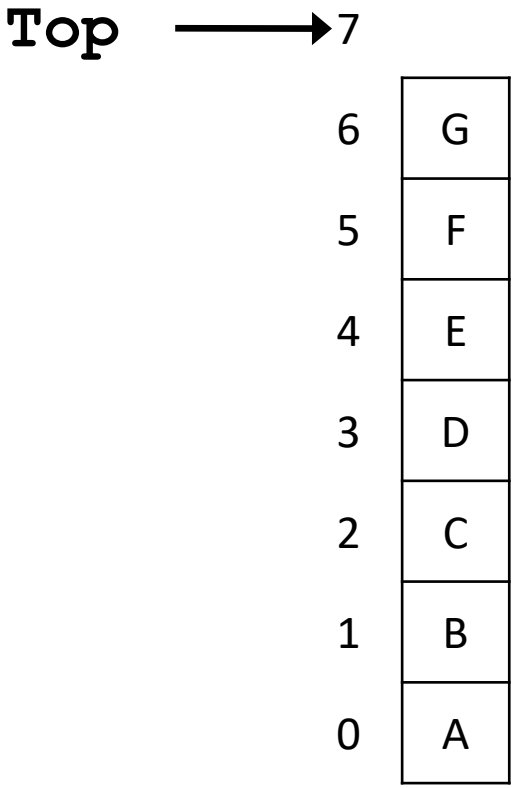
Considering **`MaxSize`** = 7
There are 4 elements inside Stack
So element will be popped from index 3

```
bool IsEmpty( void ){
      // returns True if stack has no element
      return (Top == 0);
}
```

Considering **MaxSize** = 7

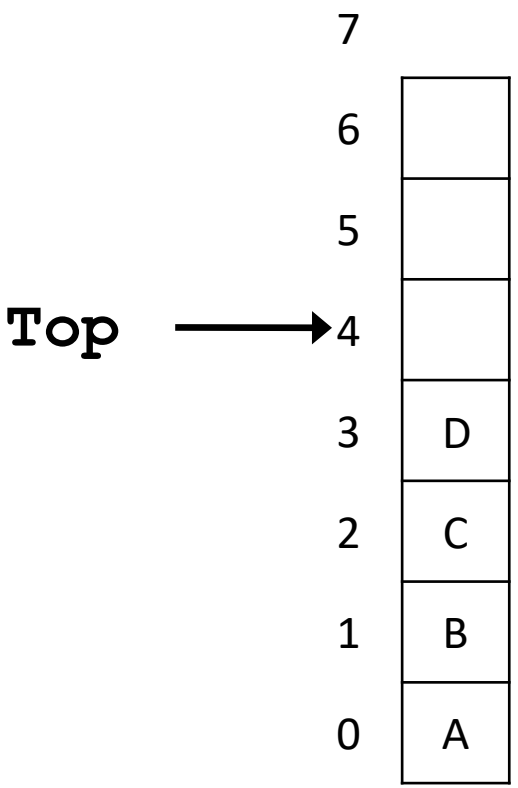


```
bool IsFull( void ){
      // returns True if stack full
      return ( Top == MaxSize );
}
```



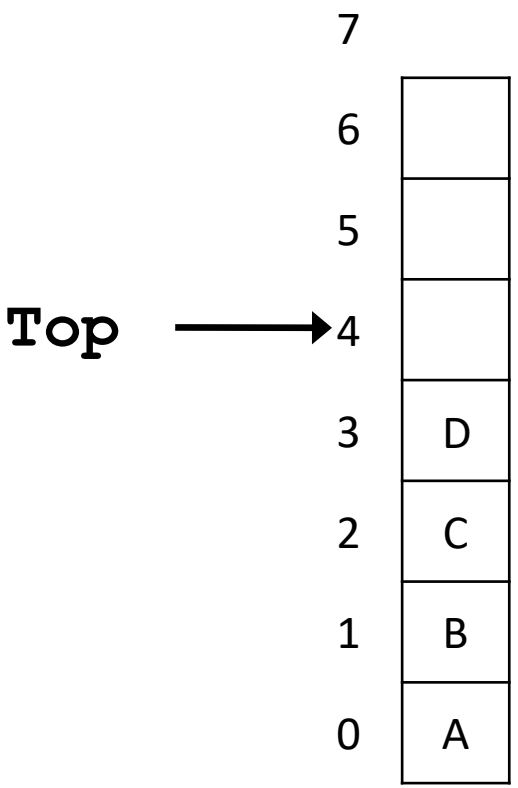
Considering **MaxSize** = 7

```
bool TopElement( int &Element ){
      // gives the top element in Element
      if( IsEmpty() ) { cout << "Stack empty\n"; return false; }
      Element = Stack[ Top - 1 ];
      return true;
}
```



Considering **MaxSize** = 7
There are 4 elements inside Stack
So top element will be at index 3

```
void Show( void ){
      // prints the whole stack
      if( IsEmpty() ) { cout << "Stack empty\n"; return; }
      for( int i=Top-1; i>=0; i-- ) cout << Stack[i] << endl;
}
```



Considering **MaxSize** = 7
There are 4 elements inside Stack
So element will be shown from index 3 down to index 0.

Array and Stack

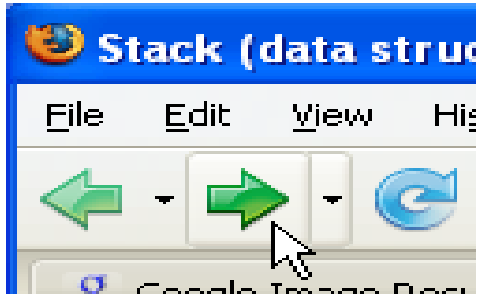
Differences…

Insertion/deletion of an element

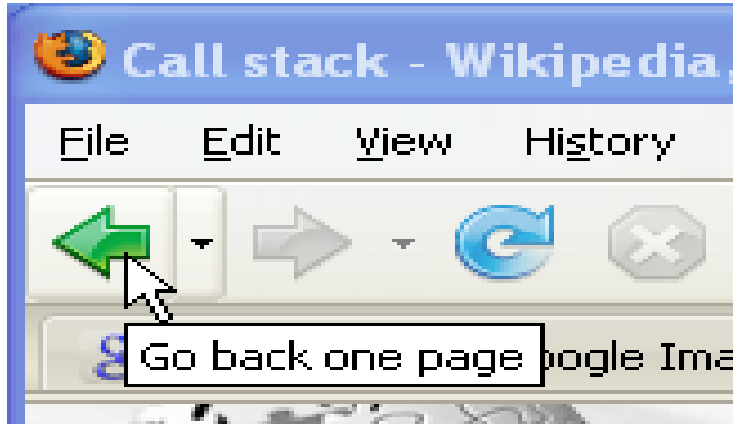
Accessing an element

STACK APPLICATIONS

- The runtime stack used by a process (running program) to keep track of methods in progress
- Search problems
- Undo, redo, back, forward
- **Arithmetic Expressions**







STACK APPLICATIONS

Arithmetic Expression

- Infix, Prefix and Postfix Notation
- Conversion of Expression
- Evaluation of Expression

STACK APPLICATIONS

Infix, Prefix and Postfix Notation

- We are accustomed to write arithmetic expressions with the operation between the two operands: a+b or c/d. If we write a+b*c, however, we have to apply precedence rules to avoid the ambiguous evaluation (add first or multiply first?).
- There's no real reason to put the operation between the variables or values. They can just as well precede or follow the operands. You should note the advantage of prefix and postfix: the need for precedence rules and parentheses are eliminated.
- Postfix expressions are easily evaluated with the aid of a stack.

Infix	Prefix	Postfix
a + b	+ a b	a b +
a + b * c	+ a * b c	a b c * +
(a + b) * (c - d)	* + a b - c d	a b + c d - *

STACK APPLICATIONS

Infix to Postfix Conversion - Algorithm

1. Create an empty stack and an empty postfix output string/stream
2. Scan the infix input string/stream left to right
3. If the current input token is an operand, simply append it to the output string
4. If the current input token is an operator, pop off all operators that have equal or higher precedence and append them to the output string; push the operator onto the stack. The order of popping is the order in the output.
5. If the current input token is '(', push it onto the stack
6. If the current input token is ')', pop off all operators and append them to the output string until a '(' is popped; discard the '('.
7. If the end of the input string is found, pop all operators and append them to the output string.

STACK APPLICATIONS

Infix to Postfix Conversion

Infix: a + (b + c) * d – e * f

Symbol	Stack	Postfix
a		a
+	+	a
(	+ (	a
b	+ (	a b
+	+ (+	a b
c	+ (+	a b c
)	+	a b c +
*	+ *	a b c +
d	+ *	a b c + d
-	-	a b c + d * +
e	-	a b c + d * + e
*	- *	a b c + d * + e
f	- *	a b c + d * + e f
		a b c + d * + e f * -

STACK APPLICATIONS

Infix to Postfix Conversion: Example

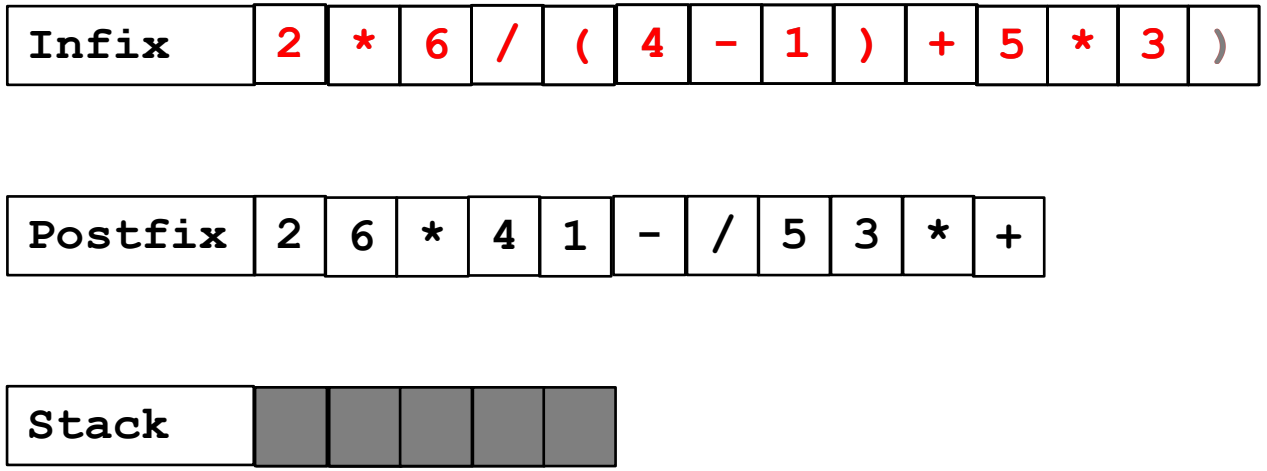
Infix: a + (b + c) * d – e * f



Postfix: a b c + d * + e f * –

Stack: + (+ * - *

CONVERTING INFIX TO POSTFIX



STACK APPLICATIONS

Evaluation of Postfix Expression - Algorithm

1) Create a stack to store operands (or values).

2) Scan the given expression and do following for every scanned element.

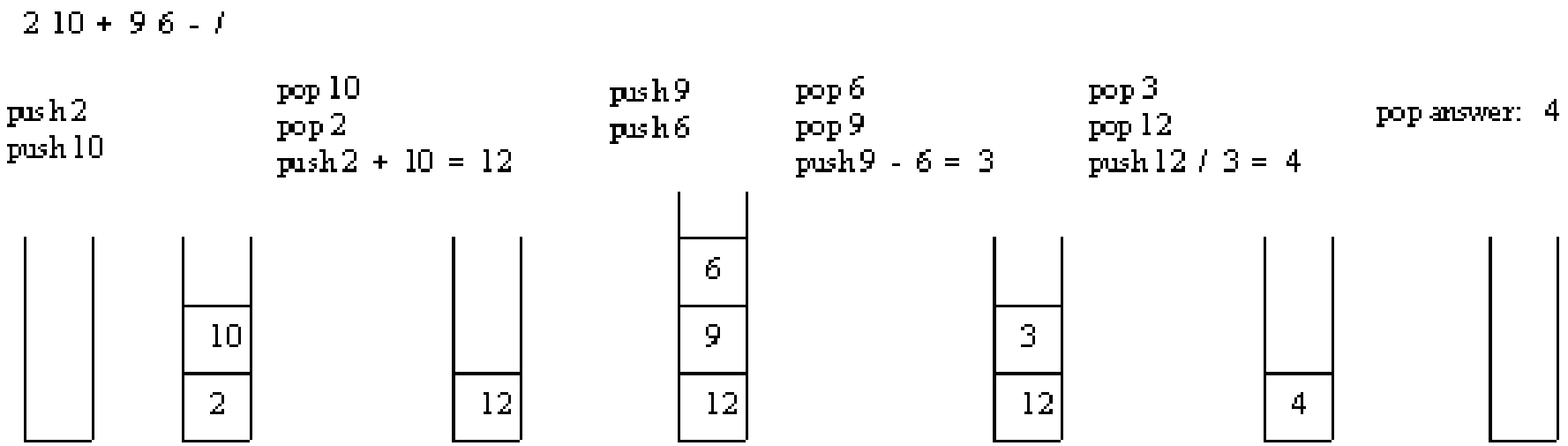
…..a) If the element is a operand, push it into the stack

…..b) If the element is a operator, pop operands for the operator from stack. Evaluate the operator and push the result back to the stack

3) When the expression is ended, the number in the stack is the final answer

STACK APPLICATIONS

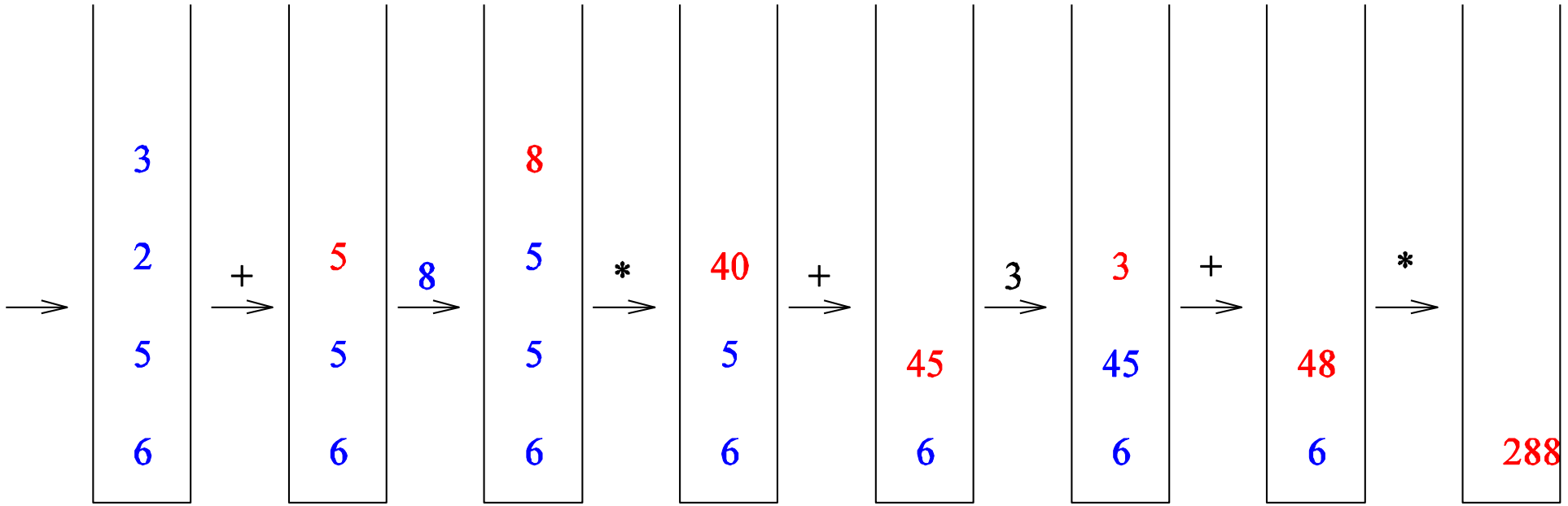
Evaluation of Postfix Expression (Example)



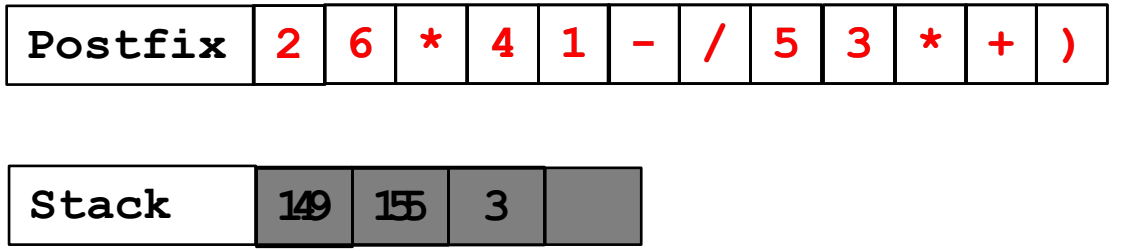
STACK APPLICATIONS

Evaluation of Postfix Expression (Example)

Expression: 6 5 2 3 + 8 * + 3 + *



EVALUATING POSTFIX EXPRESSION



STACK APPLICATIONS

Exercise

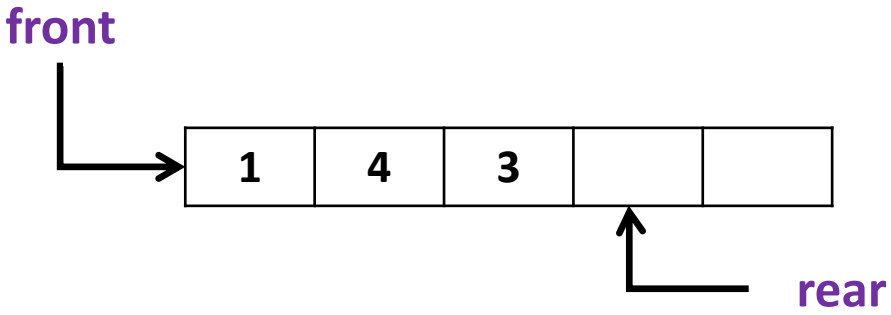
Infix: 8+(3+11)*4-3*10+7

Convert the infix operation into postfix expression and then evaluate the postfix expression using stacks.

QUEUE

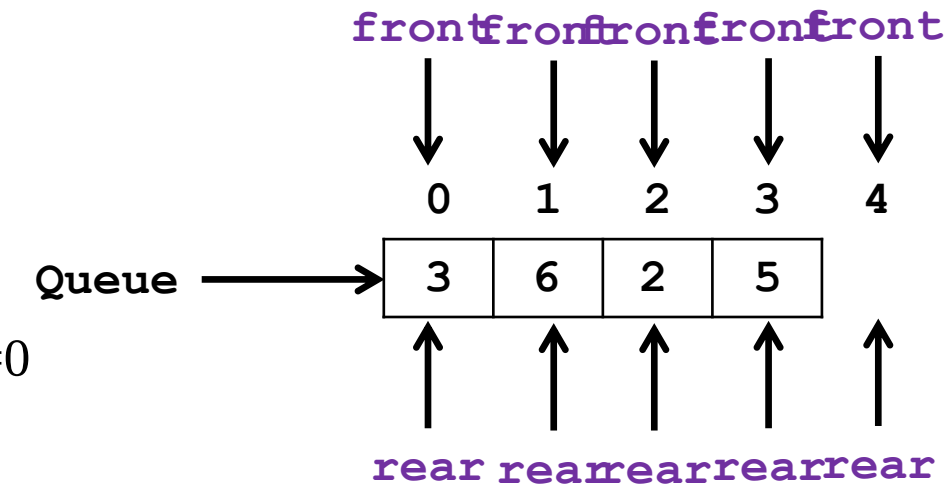
- A queue is a waiting line – seen in daily life
- A line of people waiting for a bank teller
- A line of cars at a toll both
- Queue data structure is like a container with both end opening.
  - An end called **rear**
  - Another end is **front**
- This mechanism is called First-In-First-Out (FIFO).
- Some of the applications are :
  - Device queue, printer queue, keystroke queue, etc.

QUEUE IN COMPUTER LANGUAGE



- A queue is a sequence of data elements
- In the sequence
  - Items can be added only at the **rear**
  - Items can be removed only at the other end, **front**
- Basic operations
  - Check **IsEmpty**
  - Check **IsFull**
  - **EnQueue** (add element to back i.e. at the rear)
  - **DeQueue** (remove element from the front)
  - **FrontValue** (retrieve value of element from front)
  - **ShowQueue** (print all the values of stack from front to rear)

- **Queue[4],MaxSize=4;**



QUEUE - OPERATION

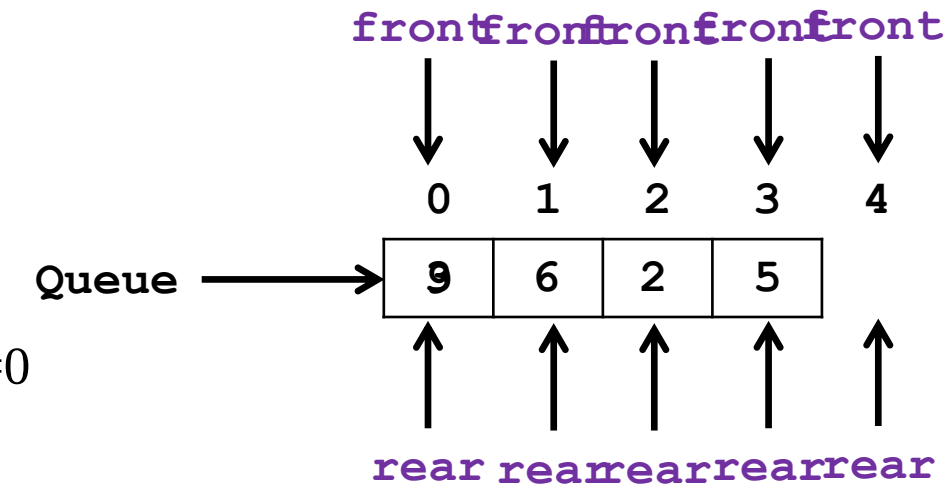
- Check **IsEmpty**
- Check **IsFull**
- **EnQueue** (add element to back i.e. at the rear)
- **DeQueue** (remove element from the front)
- **FrontValue** (retrieve value of element from front)
- **ShowQueue** (print all the values of stack from front to rear)

- Initialize( ) → front=rear=0
- EnQueue(3)
- EnQueue(6)
- EnQueue(2)
- EnQueue(5)
- EnQueue(9) → Queue Full (rear==MaxSize)
- DeQueue(&v) → 3
- DeQueue(&v) → 6
- DeQueue(&v) → 2
- DeQueue(&v) → 5
- DeQueue(&v) → Queue Empty (front==rear)

Circular Queue - Operation

- Check **IsEmpty**
- Check **IsFull**
- **EnQueue** (add element to back i.e. at the rear)
- **DeQueue** (remove element from the front)
- **FrontValue** (retrieve value of element from front)
- **ShowQueue** (print all the values of stack from front to rear)

- **Queue[4],MaxSize=4;**



- Initialize( ) → front=rear=0
- EnQueue(3)
- EnQueue(6)
- EnQueue(2)
- DeQueue(&v) → 3
- DeQueue(&v) → 6
- EnQueue(5) → (rear==MaxSize)→rear=0
- EnQueue(9)
- DeQueue(&v) → 2
- DeQueue(&v) → 5 (front==MaxSize) → front=0
- DeQueue(&v) → 9
- DeQueue(&v) → Queue Empty (front==rear)

QUEUE

Write EnQueue and DeQueue functions for a Linear Queue

ARRAY, STACK AND QUEUE

Differences….